

```

import json

class VAPContextEngine:
    """
    Simulates the 'Contextual Override' layer of the Vibe Audio
    Protocol.
    This engine filters and modifies track metadata based on active
    scenarios.
    """

    def __init__(self):
        self.active_context = None
        self.override_rules = {
            "GYM_PEAK": {
                "min_bpm": 120,
                "min_arousal": 0.7,
                "min_met": 6.0,
                "lighting_modifier": "FORCE_RED_SHIFT",
                "narrative_filter": ["relax", "sleep", "sad"]
            },
            "NIGHT_DRIVE": {
                "max_bpm": 130,
                "min_valence": -0.5,
                "lighting_modifier": "FORCE_COOL_SHIFT",
                "narrative_filter": ["hype", "scream"]
            }
        }

    def set_context(self, context_tag):
        if context_tag in self.override_rules:
            self.active_context = context_tag
            print(f"STATUS: Context set to [{context_tag}]")
        else:
            print("ERROR: Unknown Context")

    def apply_override(self, track_vap):
        """
        Evaluates a track's V.A.P. profile against the active context
        rules.
        Returns a modified 'Contextual Profile' with compatibility
        scores.
        """
        if not self.active_context:
            return {"status": "PASSTHROUGH", "compatibility": 1.0,
                    "data": track_vap}

        rules = self.override_rules[self.active_context]
        score = 1.0

```

```

modifications = []
rejection_reasons = []

# 1. KINETIC CHECK (Heart Rate / BPM)
track_bpm = track_vap['PILLARS']['STRUCTURAL']['BPM']
track_met = track_vap['PILLARS']['KINETIC']['MET_SCORE']

if 'min_bpm' in rules and track_bpm < rules['min_bpm']:
    score -= 0.4
    rejection_reasons.append(f"BPM {track_bpm} too low for
{self.active_context}")

if 'min_met' in rules and track_met < rules['min_met']:
    score -= 0.3
    rejection_reasons.append(f"MET Score {track_met}
insufficient for workout")

# 2. AFFECTIVE CHECK (Energy/Arousal)
track_arousal = track_vap['PILLARS']['AFFECTIVE']['AROUSAL']
if 'min_arousal' in rules and track_arousal <
rules['min_arousal']:
    score -= 0.3
    rejection_reasons.append("Energy level too low")

# 3. PHOTOMETRIC OVERRIDE (Lighting)
# If the track matches the vibe, we optimize the lighting for
the context
final_hex = track_vap['PILLARS']['PHOTOMETRIC']['PRIMARY_HEX']
if rules['lighting_modifier'] == "FORCE_RED_SHIFT":
    # Override cool colors to Red/Orange for intensity
    if track_vap['PILLARS']['PHOTOMETRIC']['PALETTE_TEMP'] ==
"Cool":
        final_hex = "#FF4500" # OrangeRed Override
        modifications.append("Lighting forced to High-Energy
Red")

# 4. FINAL VERDICT
status = "ACCEPTED"
if score < 0.6:
    status = "REJECTED"
elif score < 0.8:
    status = "BORDERLINE"

return {
    "context": self.active_context,
    "track_title": track_vap['IDENTITY']['TITLE'],
    "status": status,
    "compatibility_score": round(max(0, score), 2),

```

```

        "overrides_applied": modifications,
        "rejection_log": rejection_reasons,
        "output_vap": {
            "final_hex": final_hex,
            "target_hr": rules.get('min_bpm', 0)
        }
    }

# --- TEST DATA ---

# Track 1: High Energy (Matches Gym)
track_a = {
    "IDENTITY": {"TITLE": "Work", "ARTIST": "Bear Grillz"},
    "PILLARS": {
        "STRUCTURAL": {"BPM": 150},
        "AFFECTIVE": {"AROUSAL": 0.95, "VALENCE": 0.1},
        "KINETIC": {"MET_SCORE": 9.0}, # Sprint
        "PHOTOMETRIC": {"PRIMARY_HEX": "#FF0000", "PALETTE_TEMP":
"Warm"}
    }
}

# Track 2: Low Energy (Clashes with Gym)
track_b = {
    "IDENTITY": {"TITLE": "Dreams", "ARTIST": "The Cranberries"},
    "PILLARS": {
        "STRUCTURAL": {"BPM": 128}, # Technically fast enough, but...
        "AFFECTIVE": {"AROUSAL": 0.4, "VALENCE": 0.6}, # ...energy is
low
        "KINETIC": {"MET_SCORE": 4.0}, # Moderate/Walk
        "PHOTOMETRIC": {"PRIMARY_HEX": "#00FFFF", "PALETTE_TEMP":
"Cool"}
    }
}

# --- RUN SIMULATION ---

engine = VAPContextEngine()

print("--- SIMULATION START: CONTEXT [GYM_PEAK] ---")
engine.set_context("GYM_PEAK")
print("\n")

# Process Track A
result_a = engine.apply_override(track_a)
print(f"TRACK: {result_a['track_title']}")
print(json.dumps(result_a, indent=2))
print("\n" + "-"*30 + "\n")

```

```
# Process Track B
result_b = engine.apply_override(track_b)
print(f"TRACK: {result_b['track_title']}")
print(json.dumps(result_b, indent=2))
```